

SPARTACUS GYM

Sistema Integral de Gestión de Membresías, Control de Turnos y Analítica Operativa.

2. Descripción General

Spartacus GYM es un sistema web *Full-Stack* concebido para modernizar la gestión administrativa de centros de acondicionamiento físico. La solución sustituye los registros manuales y planillas de cálculo descentralizadas por una arquitectura cliente-servidor automatizada. El sistema centraliza la administración de miembros, procesa el estado de vigencia de sus membresías en tiempo real, clasifica a los usuarios por franjas horarias de asistencia y genera métricas analíticas para la toma de decisiones operativas por parte del personal de recepción y administración.

3. Objetivo General

Desarrollar e implementar un sistema web integral que automatice el control de acceso, la facturación de planes y la distribución de flujo de personas en un gimnasio, garantizando el cálculo preciso del estado de cada suscripción, la optimización de búsquedas de socios y la visualización centralizada de indicadores clave de rendimiento (KPIs).

4. Alcance del Sistema

El sistema cubre el ciclo operativo completo del área de recepción y administración del gimnasio:

- **Módulo de Miembros:** Registro completo (Alta), actualización de datos personales y de contacto (Modificación), desactivación/eliminación (Baja) y consulta parametrizada.
- **Módulo de Planes y Tarifas:** Administración de suscripciones predefinidas (Pase Libre, Clases Grupales, Pase Diario, Musculación Anual) con duraciones y precios estandarizados.
- **Motor de Vigencias:** Clasificación en tiempo real de cada miembro (*VIGENTE*, *VENCIDO*, *SIN PLAN*) mediante la comparación de la fecha actual frente al vencimiento registrado.
- **Módulo de Control de Turnos:** Categorización de la asistencia por franjas (*MAÑANA*, *TARDE*, *NOCHE*) para evitar la saturación de las instalaciones.
- **Dashboard Analítico:** Panel interactivo que calcula el total de socios activos y la distribución porcentual de miembros por turno.
- **Límites del alcance:** La versión actual se enfoca de manera exclusiva en el panel administrativo (*Backoffice*). No incluye pasarela de pagos automatizada de terceros ni aplicación móvil nativa para el cliente final.

5. Tecnologías Utilizadas

Capa / Dominio	Tecnología / Herramienta	Función / Descripción
Backend Framework	Java 25 / Spring Boot 3.x	Núcleo de la API RESTful, inversión de control y gestión de dependencias.
Construcción Backend	Gradle (`build.gradle`, `gradlew`)	Automatización de compilación, gestión de dependencias y empaquetado JAR.
ORM / Persistencia	Spring Data JPA / Hibernate	Mapeo objeto-relacional de entidades a tablas MySQL sin código SQL repetitivo.
Base de Datos	MySQL Server 8.x	Motor relacional para el almacenamiento persistente y transaccional.
Frontend Framework	Angular 19+ (Standalone)	Desarrollo de la SPA (*Single Page Application*) reactiva y modular.
Lenguaje Frontend	TypeScript / HTML5 / CSS3	Lógica tipada de cliente, maquetación semántica y hojas de estilo estructuradas.
Entorno Frontend	Node.js / npm	Gestor de paquetes y ejecutor del servidor de desarrollo de Angular.
IDE de Desarrollo	Spring Tools Suite (STS) & VS Code	Entornos de desarrollo integrados para Backend (Java) y Frontend (Web).
Control de Versiones	Git & GitHub	Control de código fuente distribuido con estrategia de repositorios independientes.

6. Funcionalidades Principales

1. **Evaluación Dinámica de Suscripciones:** Algoritmo que calcula automáticamente si el socio está al día analizando el rango de fechas (`fecha_vencimiento >= LocalDate.now()`).
2. **Buscador Multicriterio Optimizado:** Filtro reactivo que permite localizar clientes combinando campos de texto (Nombre, Apellido o DNI) cruzados condicionalmente con el turno seleccionado.
3. **Procesamiento de Métricas Operativas:** Endpoints especializados que computan agregaciones cuantitativas (recuento de miembros activos vs. inactivos y densidad de ocupación por turno).
4. **Gestión Centralizada de Excepciones:** Mecanismo de captura global de errores que transforma fallos de servidor o datos no encontrados en respuestas HTTP estandarizadas.

7. Requisitos Funcionales y No Funcionales

Requisitos Funcionales (RF)

- **RF-01:** El sistema debe permitir registrar nuevos miembros capturando obligatoriamente DNI, Nombre, Apellido, Edad, Turno y Método de Pago.
- **RF-02:** El sistema debe permitir asociar un miembro con un Plan activo y calcular de forma automática la fecha de vencimiento sumando los días de duración del plan a la fecha de asignación.
- **RF-03:** El sistema debe permitir consultar la lista total de miembros o filtrar dinámicamente por término de búsqueda (DNI/Nombre/Apellido) y turno.
- **RF-04:** El sistema debe permitir actualizar la información de un miembro o reasignar su plan de entrenamiento.
- **RF-05:** El sistema debe permitir eliminar la ficha de un miembro de la base de datos.
- **RF-06:** El sistema debe exponer métricas resumidas para el Dashboard (cantidad de socios vigentes y distribución por turno).

Requisitos No Funcionales (RNF)

- **RNF-01 (Desacoplamiento):** La arquitectura debe mantener una separación total entre la capa de presentación (Angular) y la capa de datos (Spring Boot) mediante una API REST sobre HTTPS/HTTP.
- **RNF-02 (Mantenibilidad y Limpieza):** El código debe seguir los principios SOLID, manteniendo patrones DTO y repositorios segregados, libre de archivos de configuración de IDE en el control de versiones.
- **RNF-03 (Manejo de CORS):** La API REST debe estar configurada expresamente para permitir solicitudes cruzadas (*Cross-Origin Resource Sharing*) provenientes del puerto de desarrollo de Angular.

- **RNF-04 (Integridad de Datos):** La base de datos debe aplicar restricciones de clave foránea (*Foreign Keys*) para garantizar la consistencia referencial entre miembros y planes.

8. Arquitectura del Sistema: Proceso de Construcción desde Cero

8.1. Desarrollo del Backend (Spring Boot)

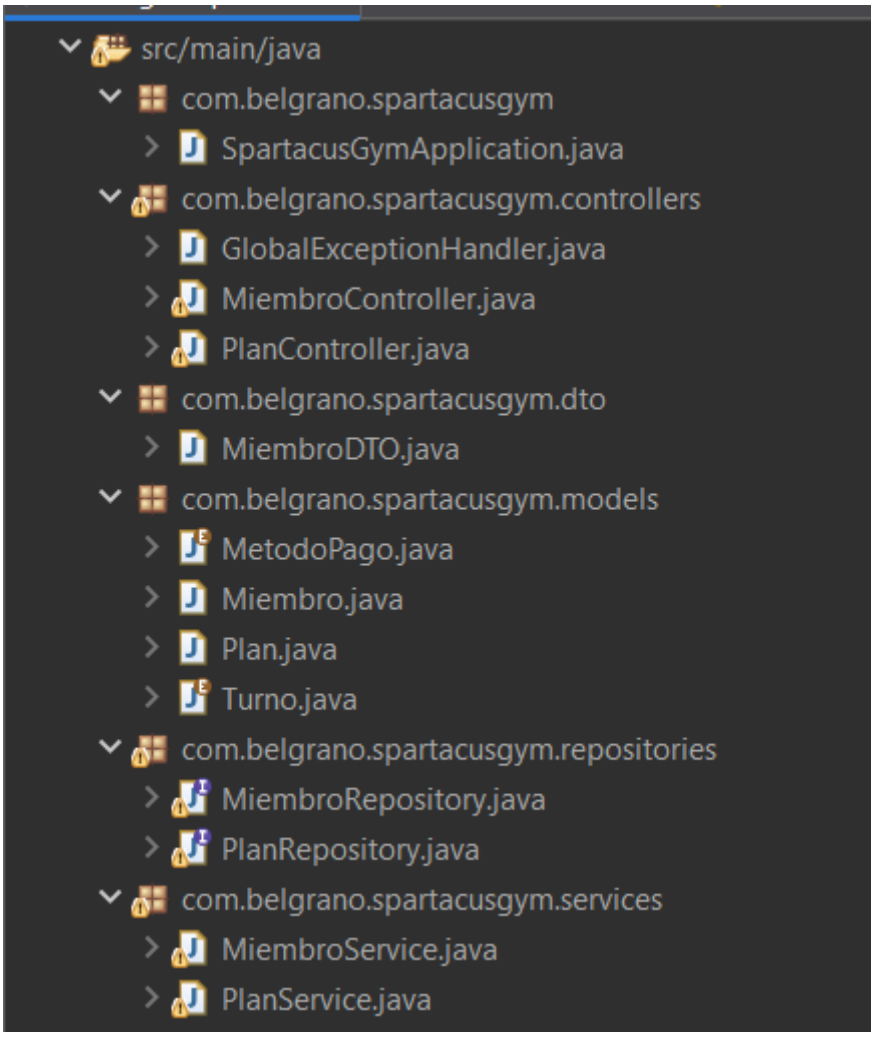
A. Inicialización y Configuración de Proyecto

El desarrollo backend se inició en **Spring Tools Suite (STS)** generando un proyecto Spring Boot con sistema de construcción **Gradle**.

- **Archivos de Construcción:** `build.gradle` define el JDK objetivo (Java 25) y las dependencias fundamentales: `spring-boot-starter-web` (para controladores REST), `spring-boot-starter-data-jpa` (para persistencia) y `mysql-connector-j` (driver JDBC). `settings.gradle` establece el nombre del módulo (`SpartacusGYM`), y los ejecutables `gradlew` / `gradlew.bat` permiten compilar el proyecto en cualquier entorno sin requerir instalación previa de Gradle.
- **Configuración de Propiedades (`src/main/resources/application.properties`):** Se configuró la URL de conexión JDBC a MySQL (`jdbc:mysql://localhost:3306/spartacus_gym`), el usuario, la contraseña y la propiedad Hibernate `spring.jpa.hibernate.ddl-auto=update`, lo que permite sincronizar las entidades Java con el motor de base de datos.

B. Estructura de Paquetes y Fundamentación de Diseño

El backend se organizó modularmente bajo la raíz `com.belgrano.spartacusgym`, dividiendo el código en capas con responsabilidades delimitadas (Principio de Responsabilidad Única):



1. **Paquete Raíz (`com.belgrano.spartacusgym`):**
   - `SpartacusGymApplication.java`: Contiene el método `main` y la anotación `@SpringBootApplication`. Es el punto de entrada que arranca el contenedor de Inversión de Control (IoC) de Spring y escanea los componentes del sistema.
2. **Paquete Modelos (`models`):**
   - *Propósito:* Representar el dominio de negocio y Mapear las tablas de la base de datos MySQL mediante anotaciones JPA.
   - `Plan.java`: Clase marcada con `@Entity` y `@Table(name = "planes")`. Define la estructura de los servicios ofrecidos (`id`, `nombre`, `duracionDias`, `precio`). Se vinculó explícitamente a la tabla `planes` para evitar duplicidades de nombres en singular/plural.
   - `Miembro.java`: Entidad principal que representa al cliente (`id`, `dni`, `nombre`, `apellido`, `edad`, `fechaVencimiento`). Incluye una relación `@ManyToOne` hacia `Plan` mediante `@JoinColumn(name = "plan_id")`.
   - `Turno.java` y `MetodoPago.java`: Enums (`@Enumerated(EnumType.STRING)`). Se crearon para restringir estrictamente los valores aceptados en la base de datos (`MANANA`, `TARDE`, `NOCHE` y `EFECTIVO`, `TARJETA`, `TRANSFERENCIA`), eliminando errores de tipeo o datos inconsistentes.
3. **Paquete Repositorios (`repositories`):**
   - *Propósito:* Abstraer el acceso a la base de datos sin escribir SQL manual repetitivo.

- `MiembroRepository.java` y `PlanRepository.java`: Interfaces que extienden `JpaRepository<T, ID>`. Otorgan de manera inmediata operaciones CRUD completas (`save`, `findById`, `findAll`, `deleteById`).
- *Consultas Personalizadas:* En `MiembroRepository` se implementaron métodos con consultas **JPQL** (Java Persistence Query Language) para realizar búsquedas avanzadas que filtran simultáneamente por texto (DNI/Nombre/Apellido) y por enumeración de turno.

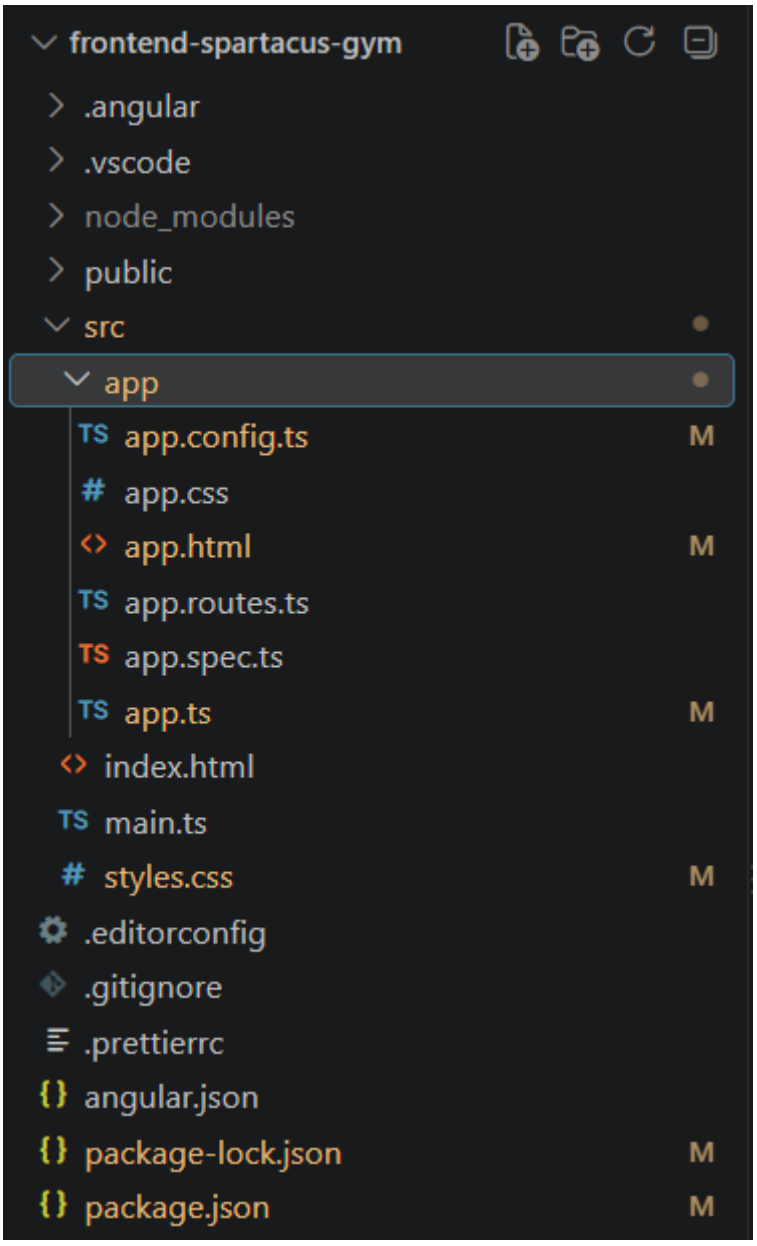
4. **Paquete Servicios (`services`):**
   - *Propósito:* Contener la lógica de negocio pura y actuar como intermediario entre los controladores y la base de datos.
   - `MiembroService.java` y `PlanService.java`: Anotadas con `@Service`. Aquí se procesan las reglas de la aplicación: el cálculo dinámico de si una membresía está *VIGENTE* o *VENCIDA* mediante `LocalDate.now()`, la asignación de planes sumando días a la fecha actual, y el cálculo de porcentajes y métricas para el Dashboard. Separa la lógica de negocio de los controladores HTTP.
5. **Paquete DTO (`dto`):**
   - *Propósito:* Desacoplar el modelo de datos interno de la representación transferida al cliente.
   - `MiembroDTO.java`: Objeto de transferencia de datos. Evita la exposición directa de las entidades de base de datos en la API REST, previene ciclos de serialización JSON infinitos al manejar relaciones bidireccionales y permite enviar campos calculados directamente al Frontend (como el estado de vigencia procesado).
6. **Paquete Controladores (`controllers`):**
   - *Propósito:* Exponer la API RESTful para responder a las solicitudes HTTP de la aplicación web.
   - `MiembroController.java` y `PlanController.java`: Anotados con `@RestController`, `@RequestMapping("/api/...")` y `@CrossOrigin(origins = "*")`. Exponen endpoints HTTP (`@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`) para que Angular pueda leer, crear, actualizar o eliminar registros.
   - `GlobalExceptionHandler.java`: Anotado con `@RestControllerAdvice`. Es un interceptor global de errores. Si se solicita un miembro inexistente o fallan las validaciones, captura la excepción y retorna una respuesta JSON limpia con códigos de estado HTTP apropiados (404 Not Found, 400 Bad Request, 500 Internal Error) en lugar de exponer stack traces de Java.

8.2. Desarrollo del Frontend (Angular)

A. Inicialización y Arquitectura Standalone

El desarrollo del cliente web se inició utilizando **Angular CLI** (`ng new frontend-spartacus-gym`). Se eligió la arquitectura **Standalone Component** (disponible en las versiones modernas de Angular), prescindiendo del archivo tradicional `app.module.ts`. Esto reduce el código repetitivo (*boilerplate*) y mejora los tiempos de carga al importar únicamente las dependencias necesarias a nivel de componente.

B. Estructura de Archivos y Responsabilidades



1. **Punto de Entrada e Inicialización:**
   - `main.ts`: Archivo principal de arranque que ejecuta la función `bootstrapApplication(AppComponent, appConfig)`. Inicia la aplicación Angular pasando el componente raíz y la configuración global.
   - `app.config.ts`: Define los proveedores globales del sistema (`ApplicationConfig`). Aquí se registra `provideHttpClient()` para habilitar el cliente de peticiones HTTP asíncronas hacia la API de Spring Boot, y `provideRouter(routes)` para el sistema de navegación.
2. **Enrutamiento y Vistas:**
   - `app.routes.ts`: Arreglo de rutas de la SPA (`Routes`). Asocia las URLs del navegador con los componentes correspondientes (Vista de Dashboard, Tabla de Administración de Miembros, Formulario de Registro/Edición).
3. **Componente Raíz y Estilos:**
   - `app.ts` (`AppComponent`): Clase TypeScript del componente principal. Gestiona el estado general de la interfaz y coordina la carga de las vistas.

- `app.html`: Plantilla HTML global que estructura el layout de la aplicación (barra de navegación superior/sidebar, contenedores principales y el `<router-outlet>` donde se renderizan las vistas dinámicas).
- `app.css`: Estilos CSS específicos del contenedor principal.
- `styles.css`: Hoja de estilos global. Define las variables CSS de color del gimnasio, tipografías, resets de márgenes y reglas para las etiquetas visuales (*badges* de estados: verde/cian para vigente, rojo para vencido, naranja para sin plan).
- `index.html`: Documento HTML5 base donde se inyecta la aplicación Angular (`<app-root>`).

4. **Archivos de Configuración del Entorno Node/Angular:**
   - `package.json` y `package-lock.json`: Especifican la lista de dependencias y librerías de Node.js necesarias para ejecutar el proyecto.
   - `angular.json`: Archivo de configuración de compilación de Angular CLI (rutas de assets, hojas de estilo globales, opciones de optimización para producción).
   - `tsconfig.json` / `tsconfig.app.json`: Configuración del compilador de TypeScript (reglas de tipado estricto, ecmascript target).

9. Base de Datos

El diseño relacional se implementó en **MySQL Server** bajo el esquema `spartacus_gym`. Consta de dos tablas vinculadas mediante integridad referencial.

Modelo Entidad-Relación (Estructura DDL)

```sql
CREATE DATABASE IF NOT EXISTS spartacus_gym;
USE spartacus_gym;

-- Tabla de Planes/Suscripciones
CREATE TABLE IF NOT EXISTS planes (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(255),
    duracion_dias INT,
    precio DECIMAL(38,2)
);

-- Tabla de Miembros
CREATE TABLE IF NOT EXISTS miembros (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    dni VARCHAR(255),
    nombre VARCHAR(255),
    apellido VARCHAR(255),
    edad INT,
    turno VARCHAR(255),
    metodo_pago VARCHAR(255),
    plan_id BIGINT,
    fecha_vencimiento DATE,
    FOREIGN KEY (plan_id) REFERENCES planes(id)
);
```

Estrategia de Carga e Inicialización (`spartacus_gym_init.sql`)

Para garantizar pruebas de desarrollo confiables, se diseñó un script de inicialización con las siguientes premisas:

1. **Creación Defensiva:** Uso de `CREATE TABLE IF NOT EXISTS` para evitar colisiones durante la auto-creación del ORM Hibernate.
2. **Sincronización de Nombres:** Nomenclatura unificada en plural (`planes`) sincronizada mediante `@Table(name = "planes")` en Java, evitando la generación de tablas duplicadas.
3. **Limpieza Previa:** Sentencias `DELETE FROM miembros;` y `DELETE FROM planes;` para permitir la re-ejecución del script sin duplicar claves primarias.
4. **Set de Datos Muestra (Mock Data):** Inserción de 4 planes base y 6 miembros representativos que contemplan los tres escenarios de prueba requeridos por la interfaz:
    - *Casos Vigentes:* Fechas de vencimiento situadas en el futuro.
    - *Caso Vencido:* Socio con fecha de vencimiento anterior a la fecha actual para validar la alerta roja.
    - *Caso Sin Plan:* Socio registrado con `plan_id = NULL` para validar la etiqueta naranja de inscripción pendiente.

10. Integrantes del Equipo y Distribución de Roles

- **Eduardo Reynaga:**

    - *Rol:* Liderazgo de Backend y Arquitectura.
    - *Responsabilidades:* Configuración del proyecto Spring Boot en STS, implementación del patrón DTO, desarrollo de la capa de Servicios y Repositorios con consultas JPQL, creación del manejador global de excepciones, despliegue de la API REST, limpieza de archivos compilados/IDE y gestión de repositorios Git/GitHub para la entrega del backend.

- **Nahuel Rivera:**

    - *Rol:* Liderazgo de Frontend y Experiencia de Usuario.
    - *Responsabilidades:* Creación y estructura del cliente web en Angular con arquitectura Standalone, maquetación HTML/CSS para el Dashboard y tablas de administración, consumo de la API REST mediante `HttpClient`, configuración de dependencias de Node/npm, prueba de rutas y subida del repositorio Frontend independiente.

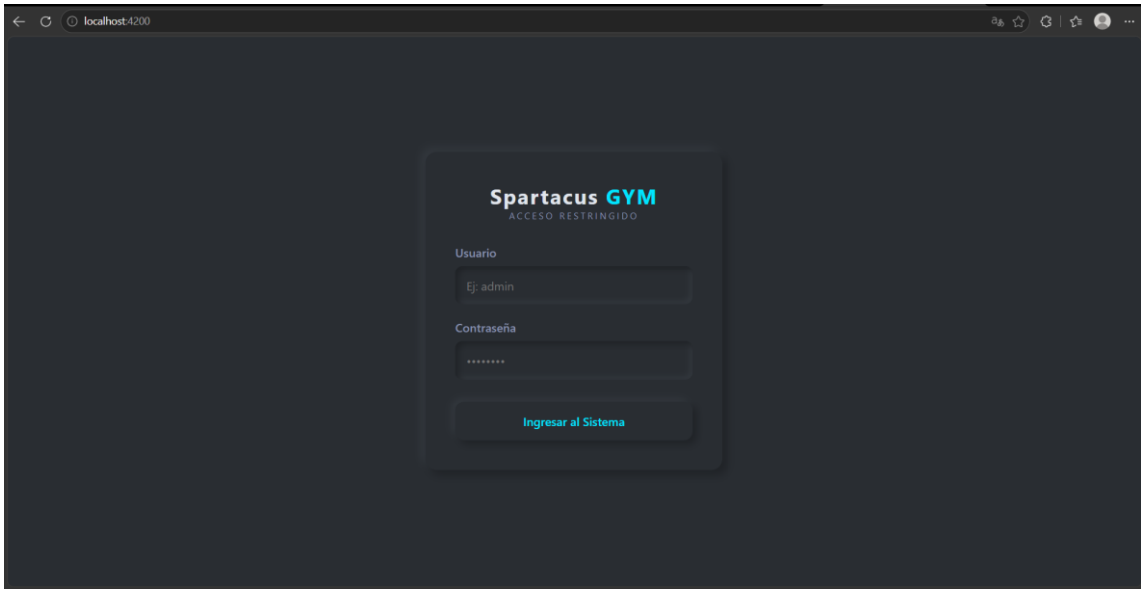
- **Gerónimo Tapia:**

    - *Rol:* Liderazgo de la Base de Datos y Documentación técnica.
    - *Responsabilidades:* Encargado de la base de datos relacional en MySQL, responsable de documentar todo el procedimiento del desarrollo del sistema, y creador del Manual de Usuario para la operación final de la plataforma.

MANUAL DE USUARIO

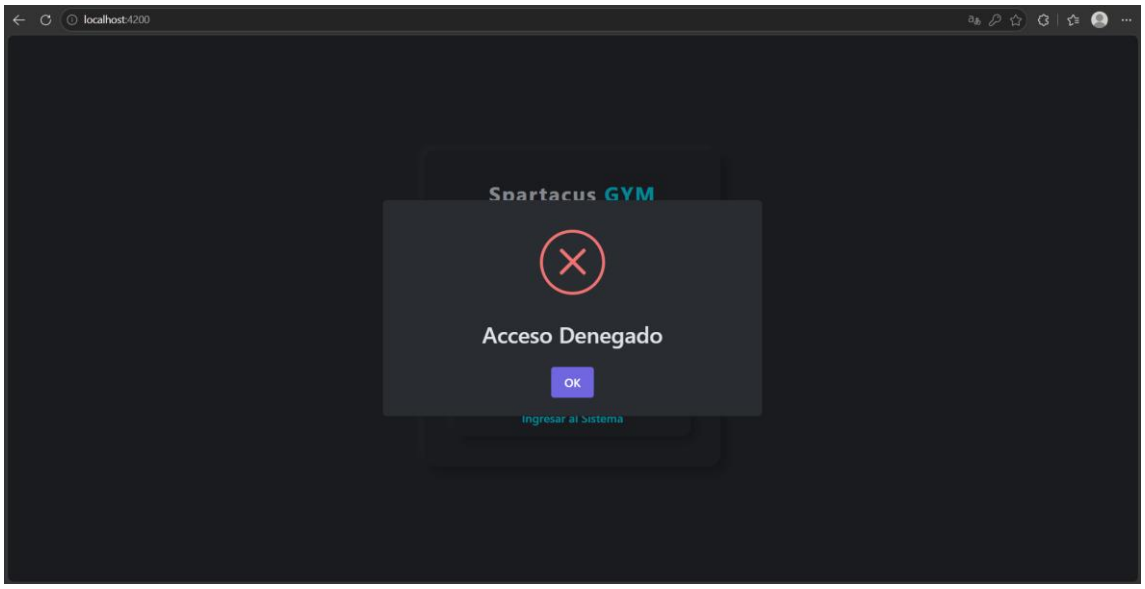
1. Acceso e Inicio de Sesión

1. Abra su navegador web (Google Chrome, Mozilla Firefox o Microsoft Edge).
2. Ingrese la dirección web local o del servidor de la plataforma: `http://localhost:4200` *(o la URL asignada a recepción)*.
3. El sistema cargará automáticamente la pantalla principal con la barra de navegación superior y el panel de control general.



Se debe ingresar los datos correctos para ingresar a la pagina principal
Usuario: admin

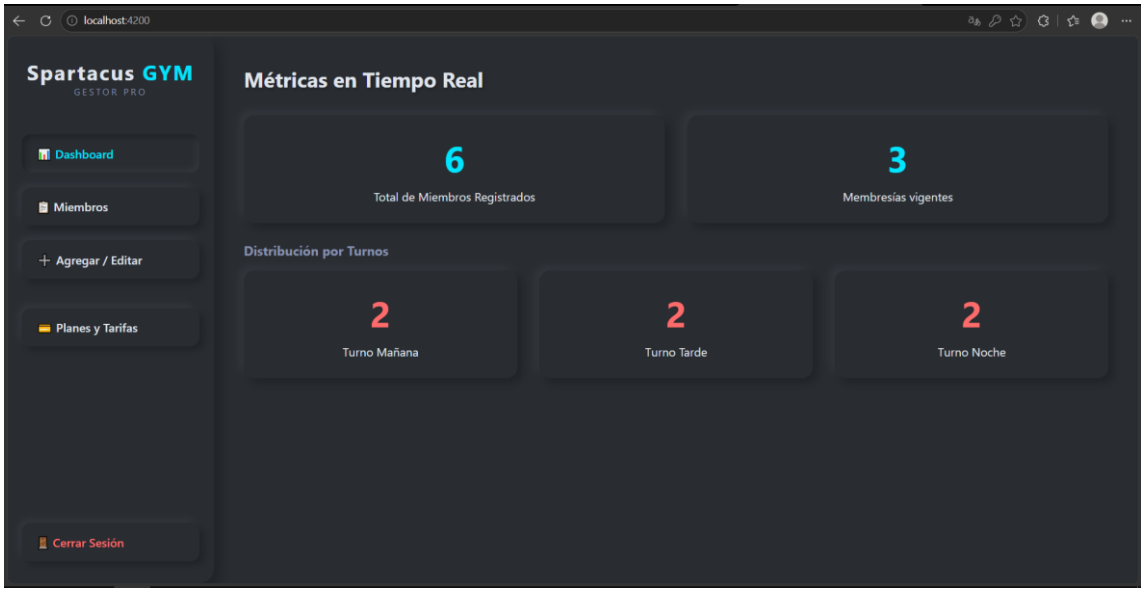
Contraseña: admin123



De lo contrario le tira error y tendrá que ingresar los datos correctamente

2. Panel Principal (Dashboard y Analítica)

El **Dashboard** le permite visualizar el estado de salud del gimnasio en pocos segundos sin necesidad de revisar manualmente socio por socio.

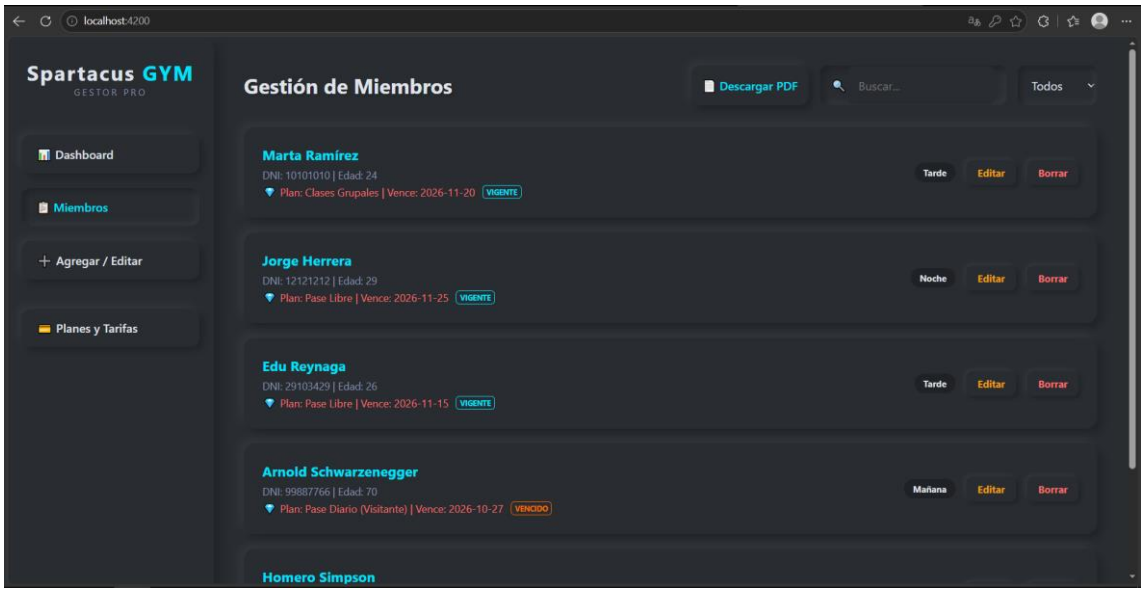


Componentes del Dashboard:

- **Métricas Globales (Tarjetas de Contador):**
  - **Total Inscriptos:** Suma total de miembros registrados en el sistema.
  - **Miembros Activos:** Cantidad exacta de clientes que tienen una membresía **VIGENTE**.
- **Distribución por Turno:** Gráfico o indicadores numéricos que muestran cuántos socios asisten en cada franja horaria (**MAÑANA**, **TARDE** o **NOCHE**).

3. Módulo de Gestión de Miembros

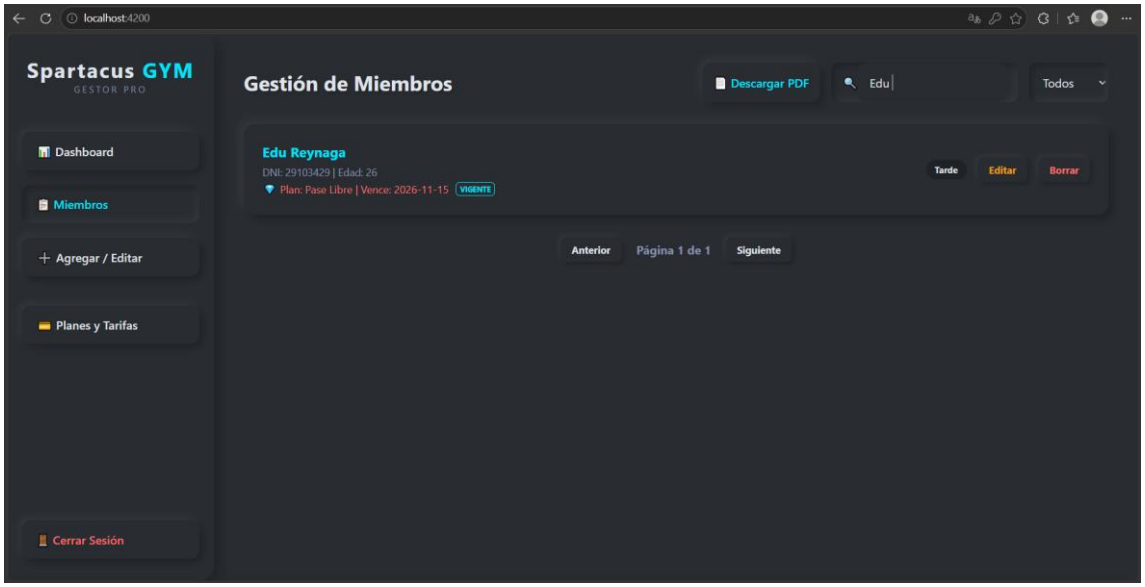
En esta sección administrará la base de datos de los clientes del gimnasio.



3.1. Consulta de Miembros y Buscador Multi-criterio

Para buscar a un socio en la tabla general:

1. Diríjase a la barra de búsqueda ubicada en la parte superior de la tabla de miembros.
2. Escriba el **DNI**, **Nombre** o **Apellido** del cliente.
3. *(Opcional)* Seleccione un filtro por **Turno** (Mañana, Tarde o Noche) en la lista desplegable.
4. La tabla se actualizará en tiempo real mostrando únicamente los coincidencias encontradas.

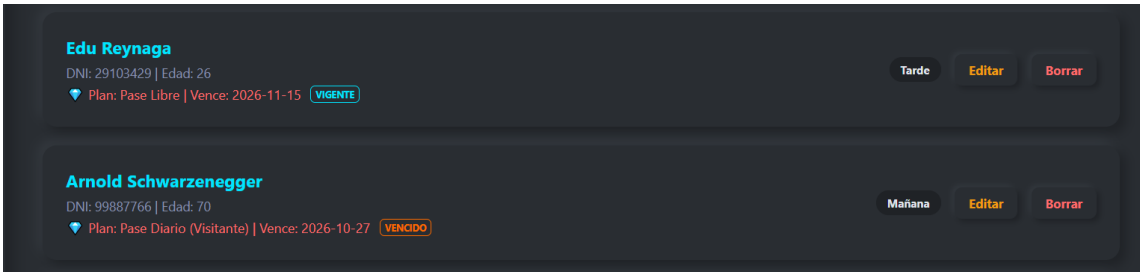


3.2. Identificación Visual de Estados de Membresía

Cada socio cuenta con una etiqueta de color (*badge*) que indica si está habilitado para ingresar al establecimiento:

- **VIGENTE (Celeste):** El socio está al día. Su fecha de vencimiento es posterior a la fecha actual.
- **VENCIDO (Naranja):** La membresía ha caducado. El socio debe abonar la renovación para ingresar a entrenar.

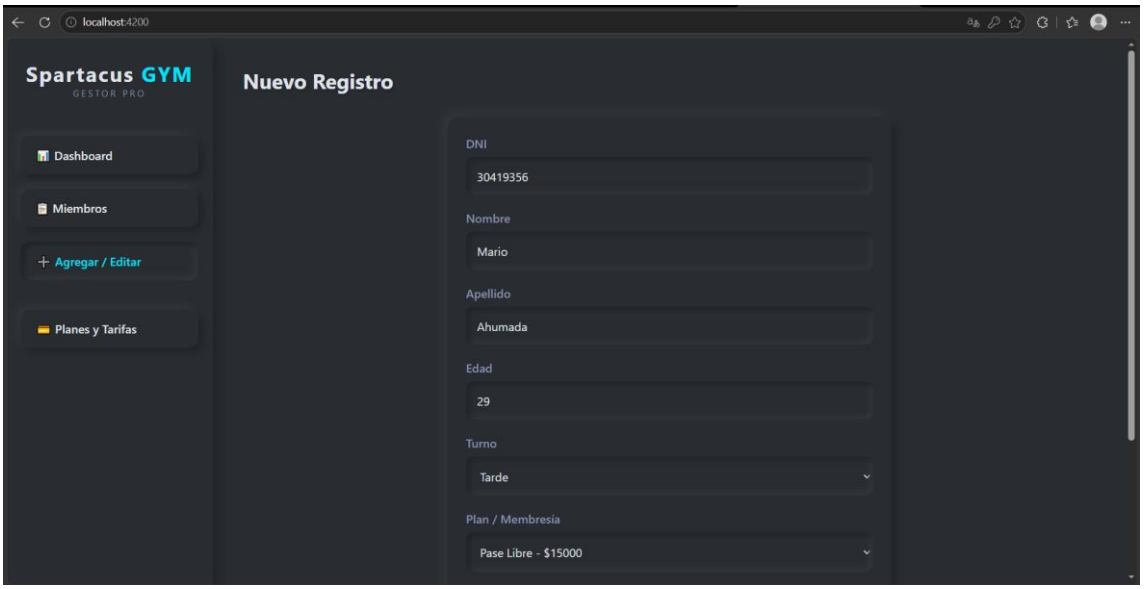
  Anotación: los que no tienen un plan asignado no tiene un cartelito asignado aún, se espera que en una futura actualización se corrija ese error

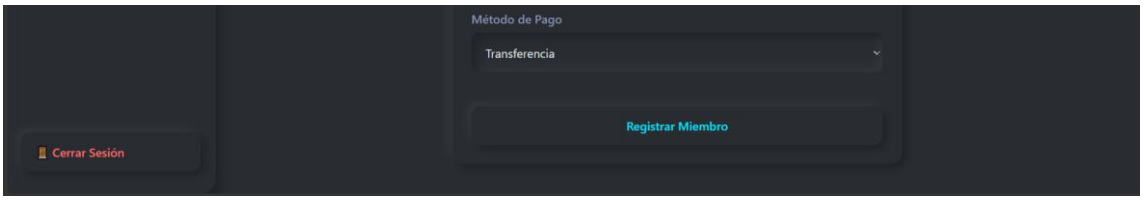


3.3. Registro de un Nuevo Miembro (Alta)

Para dar de alta a un nuevo socio en el gimnasio:

1. Haga clic en el botón verde **"+ Nuevo Miembro"** o **"Registrar Socio"**.
2. Complete el formulario con los datos obligatorios del cliente:
   - **DNI:** *(Sin puntos ni espacios).*
   - **Nombre y Apellido.**
   - **Edad.**
   - **Turno habitual:** Seleccione entre *MAÑANA*, *TARDE* o *NOCHE*.
   - **Método de Pago:** Seleccione *EFECTIVO*, *TARJETA* o *TRANSFERENCIA*.
   - **Plan de Entrenamiento:** Elija una tarifa del catálogo (ej. *Pase Libre*, *Musculación Anual*).



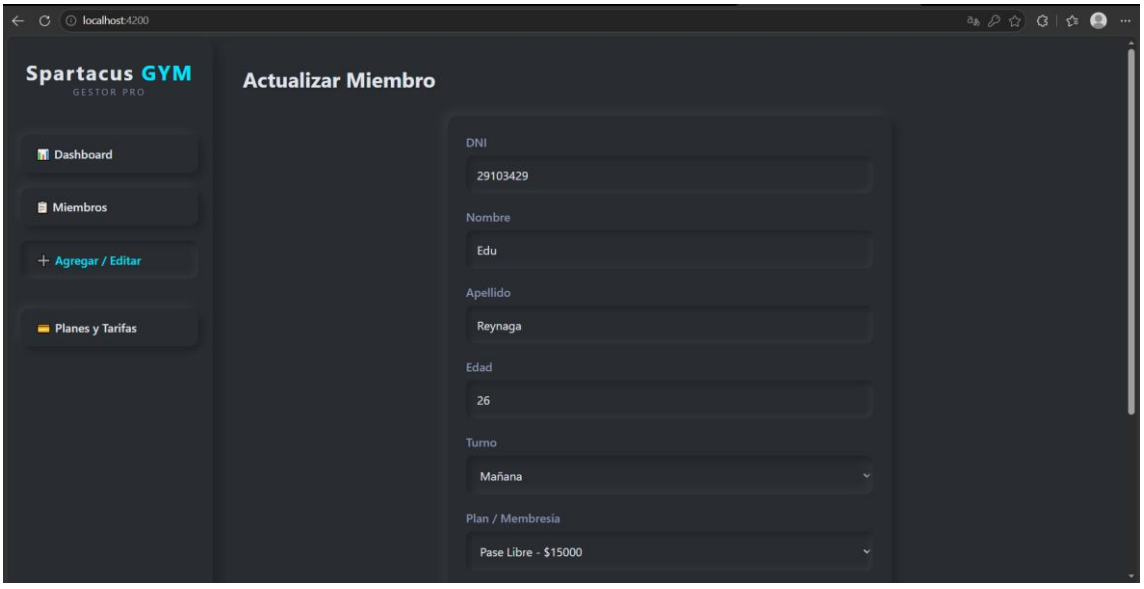


Haga clic en **"Registrar Miembro"**. El sistema asignará automáticamente la fecha de vencimiento sumando la duración del plan a la fecha de hoy.

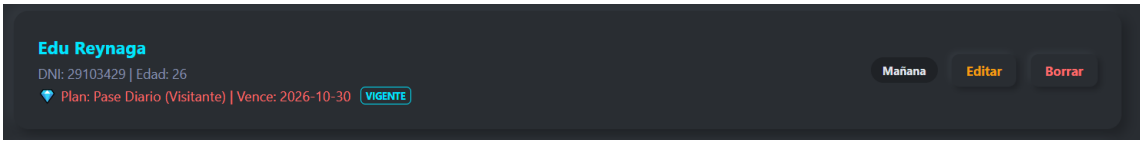
3.4. Renovación de Plan o Modificación de Datos

Cuando un socio existente se presenta a pagar su cuota o cambia de turno:

1. En la pantalla "**Miembros"** Ubique al socio en el registro utilizando el buscador.
2. Haga clic en el botón de acción **"Editar"** en la fila correspondiente.
3. Se abrirá la ficha del cliente con sus datos cargados.
4. Para renovar el plan: seleccione el nuevo plan abonado.
5. Presione **"Guardar/Renovar"**. El sistema recalculará la nueva fecha de vencimiento a partir del día de la transacción.



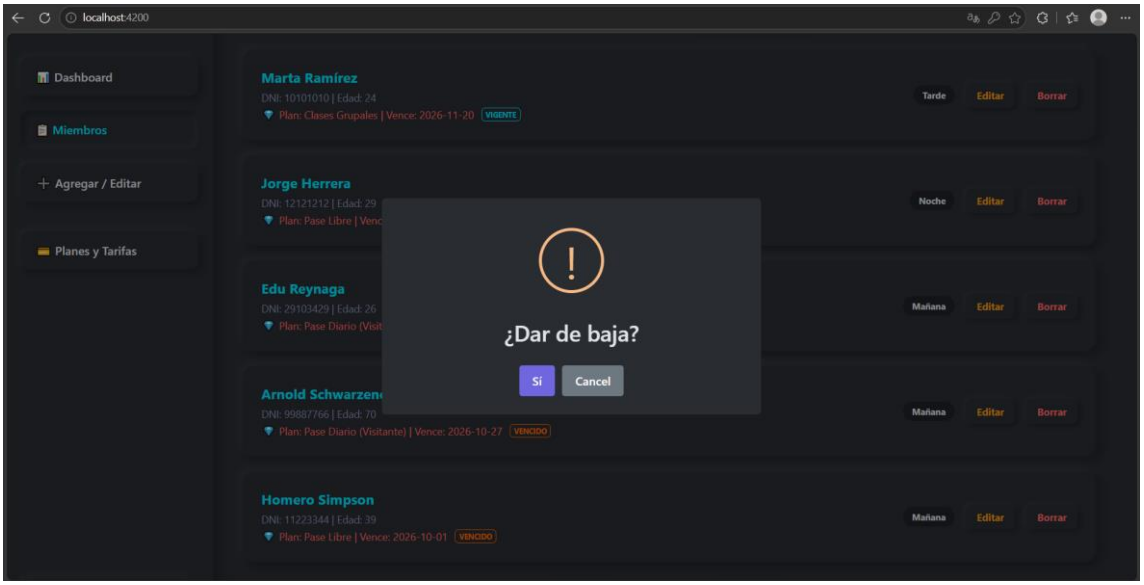




3.5. Eliminación de un Registro (Baja)

Si necesita dar de baja la ficha de un cliente:

1. Busque al socio en la tabla general.
2. Haga clic en el botón **"Eliminar"** 🗑️ *(icono de papelera).*
3. El sistema desplegará un mensaje de confirmación: *¿Está seguro de que desea eliminar a este miembro?*
4. Confirme la acción haciendo clic en **"Si"**.



4. Consulta del Catálogo de Planes y Tarifas

Para consultar las opciones de suscripción disponibles en el gimnasio:

1. Diríjase a la pestaña o sección **"Planes/Tarifas"** en el menú de navegación.
2. Visualizará el catálogo de opciones predefinidas:
    - **Pase Libre:** 30 días de acceso ilimitado.
    - **Clases Grupales:** 30 días de acceso a actividades dirigidas.
    - **Pase Diario:** 1 día de acceso promocional o visitante.

- **Musculación Anual:** 365 días de suscripción con tarifa diferencial.

